<u>Machine Learning & Deep Learning</u>

Experiment 6

Name: Abhishek Vishwakarma
Class: D15C
Roll No: 47

AIM: Apply K-Means and Hierarchical Clustering on sample datasets

1. Dataset Source

- **Dataset Name:** Mall Customer Segmentation Data
- **Source:** [Kaggle - Mall Customer Segmentation Data]

2. Dataset Description

This dataset contains data on customers of a mall. The aim is to segment customers into distinct groups based on their spending behavior and income to aid marketing strategies.

- **Size:** 200 records, 5 columns.
- **Target Variable:** None (Unsupervised Learning).
- **Features:**
    - **CustomerID** : Unique ID.
    - **Gender** : Male/Female.
    - **Age** : Age of the customer.
    - **Annual Income** (k$) : Annual Income of the customer.
    - **Spending Score (1-100)** : Score assigned by the mall based on customer behavior and spending nature.

3. Mathematical Formulation of the Algorithm

A. K-Means Clustering:

K-Means partitions n observations into k clusters in which each observation belongs to the cluster with the nearest mean (centroid).

- **Objective:** Minimize the **Within-Cluster Sum of Squares (WCSS)**, also known as Inertia.

$$J = \sum_{j=1}^{k} \sum_{i=1}^{n} ||x_i^{(j)} - c_j||^2$$

Where:

- $x_i^{(j)}$ is a data point in cluster j.
- c_j is the centroid of cluster j.
- $||...||^2$ is the squared Euclidean distance.

B. Hierarchical Clustering (Agglomerative):

This is a "bottom-up" approach.

1. Start with n clusters (each data point is its own cluster).
2. Find the two closest clusters and merge them.
3. Repeat until only one cluster remains.

- **Linkage Criteria:** Determines distance between sets of observations. **Ward's Method** is commonly used, which minimizes the variance within clusters being merged.

4. Algorithm Limitations

- **K-Means:**
  - **Specify K:** You must define the number of clusters (k) beforehand.
  - **Spherical Assumption:** Assumes clusters are spherical and of similar size. It fails with complex shapes (e.g., concentric circles).
  - **Sensitive to Outliers:** Centroids can be dragged by outliers.

- **Hierarchical:**
  - **Computationally Expensive:** Time complexity is high ($O(n^3)$), making it unsuitable for very large datasets.
  - **Irreversible:** Once a step (merge) is done, it cannot be undone.

5. Methodology / Workflow

1. **Data Loading:** Load Mall_Customers.csv.

```
# 1. Load Dataset
try:
    df = pd.read_csv('Mall_Customers.csv')
    print("Dataset loaded successfully.")
except FileNotFoundError:
    print("Error: 'Mall_Customers.csv' not found. Please upload the file.")
    df = pd.DataFrame()
```

2. **Feature Selection:** Isolate Annual Income and Spending Score for clear 2D visualization.

```
# 2. Feature Selection
# We select Annual Income (Index 3) and Spending Score (Index 4)
    X = df.iloc[:, [3, 4]].values
```

3. **K-Means Workflow:**
   - Use the **Elbow Method** (plot WCSS vs K) to find optimal K.
   - Train K-Means model.
   - Visualize clusters.

```
   # 1. Elbow Method to find optimal K
    wcss = []
    for i in range(1, 11):
        kmeans = KMeans(n_clusters=i, init='k-means++', random_state=42,
n_init=10)
        kmeans.fit(X)
        wcss.append(kmeans.inertia_)

    plt.figure(figsize=(8, 5))
    plt.plot(range(1, 11), wcss, marker='o', color='red')
    plt.title('The Elbow Method')
    plt.xlabel('Number of clusters')
    plt.ylabel('WCSS')
    plt.show()

    # 2. Train K-Means with Optimal K (Usually 5 for this dataset)
    k_optimal = 5
    kmeans = KMeans(n_clusters=k_optimal, init='k-means++', random_state=42,
n_init=10)
```

```
y_kmeans = kmeans.fit_predict(X)

# 3. Visualizing the Clusters
plt.figure(figsize=(10, 6))
colors = ['red', 'blue', 'green', 'cyan', 'magenta']
for i in range(k_optimal):
    plt.scatter(X[y_kmeans == i, 0], X[y_kmeans == i, 1],
                s=100, c=colors[i], label=f'Cluster {i+1}')

# Plot Centroids
plt.scatter(kmeans.cluster_centers_[:, 0], kmeans.cluster_centers_[:, 1],
            s=300, c='yellow', label='Centroids', edgecolors='black')
plt.title('K-Means: Clusters of Customers')
plt.xlabel('Annual Income (k$)')
plt.ylabel('Spending Score (1-100)')
plt.legend()
plt.show()
```

4. **Hierarchical Workflow:**
   - Plot a **Dendrogram** to visually determine the optimal number of clusters.
   - Train Agglomerative Clustering model.
   - Visualize clusters.

```
# ═══════════════════════════════════════
# PART B: HIERARCHICAL CLUSTERING
# ═══════════════════════════════════════
print("\n--- Part B: Hierarchical Clustering ---")

# 1. Dendrogram to find optimal clusters
plt.figure(figsize=(10, 6))
dendrogram = sch.dendrogram(sch.linkage(X, method='ward'))
plt.title('Dendrogram')
plt.xlabel('Customers')
plt.ylabel('Euclidean Distances')
plt.show()

# 2. Train Agglomerative Clustering (Using K=5 based on Dendrogram)
hc = AgglomerativeClustering(n_clusters=5, metric='euclidean',
linkage='ward')
```

```
y_hc = hc.fit_predict(X)

# 3. Visualizing the Clusters
plt.figure(figsize=(10, 6))
for i in range(5):
    plt.scatter(X[y_hc == i, 0], X[y_hc == i, 1],
                s=100, c=colors[i], label=f'Cluster {i+1}')

plt.title('Hierarchical: Clusters of Customers')
plt.xlabel('Annual Income (k$)')
plt.ylabel('Spending Score (1-100)')
plt.legend()
plt.show()
```

5. **Evaluation:** Calculate **Silhouette Score** (measures how similar an object is to its own cluster compared to other clusters).

```
# 4. Evaluation
score_kmeans = silhouette_score(X, y_kmeans)
print(f"K-Means Silhouette Score: {score_kmeans:.4f}")

# 4. Evaluation
score_hc = silhouette_score(X, y_hc)
print(f"Hierarchical Silhouette Score: {score_hc:.4f}")
```

6. Performance Analysis

- **Metric:** Silhouette Coefficient (Range: -1 to +1).
- **Interpretation:** A score closer to +1 indicates that data points are well-clustered (dense and well-separated). A score near 0 indicates overlapping clusters.

<u>Code:</u>

```
# ═══════════════════════════════════════════════════
# EXPERIMENT 6: K-MEANS & HIERARCHICAL CLUSTERING
# ═══════════════════════════════════════════════════

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.cluster import KMeans, AgglomerativeClustering
```

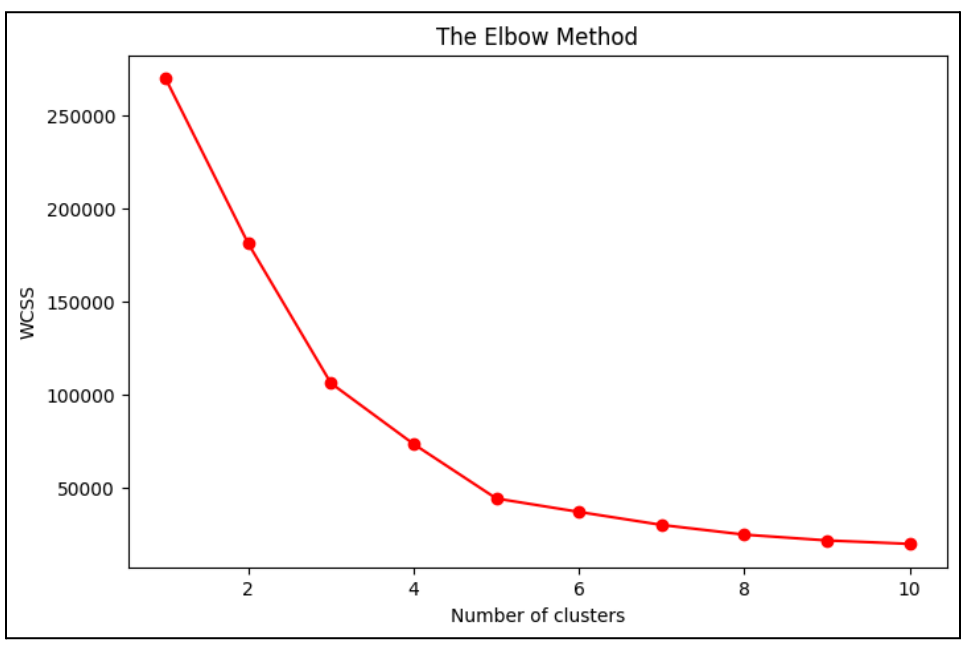
```python
from sklearn.metrics import silhouette_score
import scipy.cluster.hierarchy as sch

print("--- EXPERIMENT 6: CLUSTERING ALGORITHMS ---")

# 1. Load Dataset
try:
    df = pd.read_csv('Mall_Customers.csv')
    print("Dataset loaded successfully.")
except FileNotFoundError:
    print("Error: 'Mall_Customers.csv' not found. Please upload the file.")
    df = pd.DataFrame()

if not df.empty:
    # 2. Feature Selection
    # We select Annual Income (Index 3) and Spending Score (Index 4)
    X = df.iloc[:, [3, 4]].values

    # ==========================================
    # PART A: K-MEANS CLUSTERING
    # ==========================================
    print("\n--- Part A: K-Means Clustering ---")

    # 1. Elbow Method to find optimal K
    wcss = []
    for i in range(1, 11):
        kmeans = KMeans(n_clusters=i, init='k-means++', random_state=42, n_init=10)
        kmeans.fit(X)
        wcss.append(kmeans.inertia_)

    plt.figure(figsize=(8, 5))
    plt.plot(range(1, 11), wcss, marker='o', color='red')
    plt.title('The Elbow Method')
    plt.xlabel('Number of clusters')
    plt.ylabel('WCSS')
    plt.show()

    # 2. Train K-Means with Optimal K (Usually 5 for this dataset)
    k_optimal = 5
```

```
    kmeans = KMeans(n_clusters=k_optimal, init='k-means++', random_state=42,
n_init=10)
    y_kmeans = kmeans.fit_predict(X)

    # 3. Visualizing the Clusters
    plt.figure(figsize=(10, 6))
    colors = ['red', 'blue', 'green', 'cyan', 'magenta']
    for i in range(k_optimal):
        plt.scatter(X[y_kmeans == i, 0], X[y_kmeans == i, 1],
                    s=100, c=colors[i], label=f'Cluster {i+1}')

    # Plot Centroids
    plt.scatter(kmeans.cluster_centers_[:, 0], kmeans.cluster_centers_[:, 1],
                s=300, c='yellow', label='Centroids', edgecolors='black')
    plt.title('K-Means: Clusters of Customers')
    plt.xlabel('Annual Income (k$)')
    plt.ylabel('Spending Score (1-100)')
    plt.legend()
    plt.show()

    # 4. Evaluation
    score_kmeans = silhouette_score(X, y_kmeans)
    print(f"K-Means Silhouette Score: {score_kmeans:.4f}")


    # ==========================================
    # PART B: HIERARCHICAL CLUSTERING
    # ==========================================
    print("\n--- Part B: Hierarchical Clustering ---")

    # 1. Dendrogram to find optimal clusters
    plt.figure(figsize=(10, 6))
    dendrogram = sch.dendrogram(sch.linkage(X, method='ward'))
    plt.title('Dendrogram')
    plt.xlabel('Customers')
    plt.ylabel('Euclidean Distances')
    plt.show()

    # 2. Train Agglomerative Clustering (Using K=5 based on Dendrogram)
    hc = AgglomerativeClustering(n_clusters=5, metric='euclidean', linkage='ward')
```

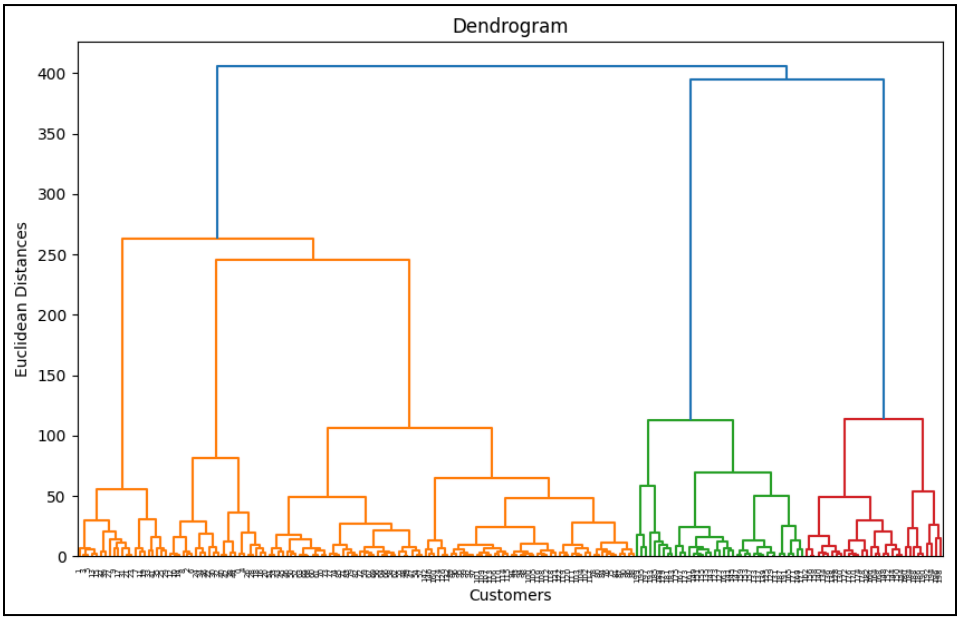
```
y_hc = hc.fit_predict(X)

# 3. Visualizing the Clusters
plt.figure(figsize=(10, 6))
for i in range(5):
    plt.scatter(X[y_hc == i, 0], X[y_hc == i, 1],
                s=100, c=colors[i], label=f'Cluster {i+1}')

plt.title('Hierarchical: Clusters of Customers')
plt.xlabel('Annual Income (k$)')
plt.ylabel('Spending Score (1-100)')
plt.legend()
plt.show()

# 4. Evaluation
score_hc = silhouette_score(X, y_hc)
print(f"Hierarchical Silhouette Score: {score_hc:.4f}")
```

7. Hyperparameter Tuning

- **K-Means:** Tuning K (Number of clusters). We look for the "elbow" point where the drop in WCSS slows down.

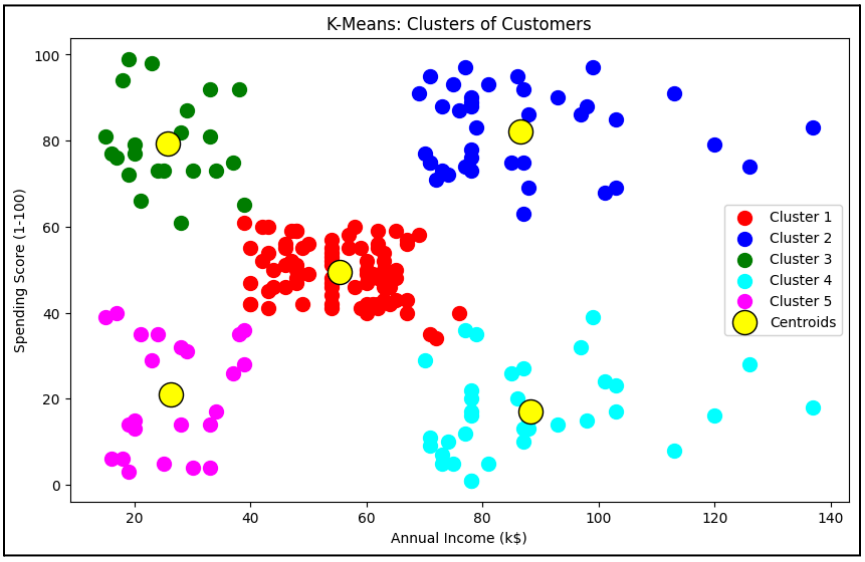


- **Hierarchical:** Tuning the "Cut-off" point on the Dendrogram to decide how many clusters to form.



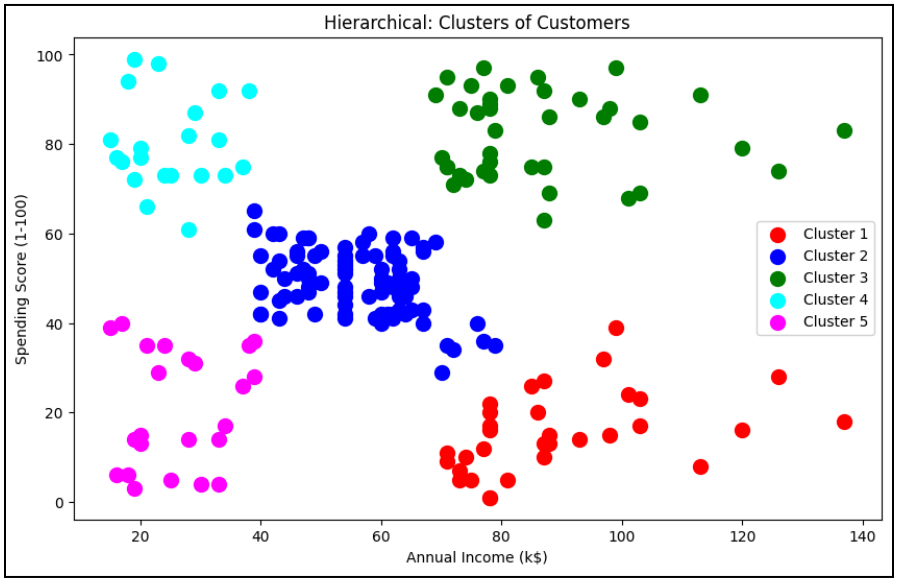
Output:

```
--- EXPERIMENT 6: CLUSTERING ALGORITHMS ---
Dataset loaded successfully.

--- Part A: K-Means Clustering —
```




```
K-Means Silhouette Score: 0.5539
```

```
--- Part B: Hierarchical Clustering —
```




```
Hierarchical Silhouette Score: 0.5530
```

8. Conclusion

In this final experiment, we applied unsupervised learning techniques to segment customers.

- **The Elbow Method** and **Dendrogram** both clearly indicated that **5** is the optimal number of clusters for this dataset.

- **Cluster Interpretation:** The algorithms successfully identified 5 distinct customer profiles:
  1. **Savers:** High Income, Low Spending.
  2. **Spenders:** High Income, High Spending (Target Audience).
  3. **Careful:** Low Income, Low Spending.
  4. **Careless:** Low Income, High Spending.
  5. **Average:** Moderate Income, Moderate Spending.

- **Comparison:** Both K-Means and Hierarchical Clustering produced nearly identical results on this dataset, as confirmed by similar Silhouette Scores. K-Means is generally preferred for larger datasets due to speed, while Hierarchical is better for understanding the nested relationships in smaller datasets.